

ExpenseLM: Distilling Policy-Aware Expense Extraction into a 4B Language Model

Suyash Kumar Singh (Roll No. 23035010145)
Term Project Report, Trimester 8
B.Sc. (Honours) Data Science and Artificial Intelligence
Indian Institute of Technology Guwahati, India
s.suyash@op.iitg.ac.in

Abstract—Converting messy, real-world expense text—chat messages, receipt OCR dumps, forwarded emails—into clean, policy-checked structured records is a common business task, today handled either manually or by calling expensive frontier-model APIs on every message. We test whether a small (~ 4 -billion parameter) open model, fine-tuned on a modest task-specific dataset, can approach frontier accuracy on this task at near-zero inference cost. We build a complete post-training pipeline: a procedurally generated, policy-labelled dataset; supervised fine-tuning (SFT) of Qwen3-4B with QLoRA; and a custom evaluation harness measuring schema validity, field accuracy, expense-level F_1 , and policy-reasoning accuracy. Fine-tuning more than doubled policy-reasoning accuracy over the base model (24.7% to 55.5%) and lifted relative-date resolution from 43% to 96%, closing roughly half the gap to a frontier ceiling using 1,600 training examples and no paid training compute. We additionally isolate and fix a concrete decoding failure mode. The study demonstrates, end to end, how a narrow capability can be distilled from a large model into a cheap, private, self-hosted one, and quantified rigorously.

Index Terms—large language models, fine-tuning, LoRA, QLoRA, structured extraction, model distillation, evaluation.

I. Introduction

Businesses receive expense information as unstructured text: a Hinglish chat message (“bhai 450 ka cab liya airport tak”), a broken receipt OCR dump, or a forwarded email with signatures and quoted threads as noise. Turning this into a clean record—amount, currency, category, merchant, date, reimbursability, and policy flags—is done either by hand or by sending every message to a frontier-model API, which is costly and moves private financial data off-site.

This mirrors one of the most common production patterns in modern applied AI: distilling a narrow capability from a large “teacher” model into a small, cheap, self-hosted “student” model [4]. The task is non-trivial because the model must not merely extract fields but apply a policy—per-category spending limits, receipt thresholds, and reimbursability rules—which requires reading comprehension and reasoning rather than pattern matching.

Objectives

- Specify the extraction task with a strict output schema and an explicit expense policy that the model must apply.
- Construct a diverse, correctly labelled dataset covering chat, OCR, email, and multilingual (English/Hinglish) inputs.
- Fine-tune a 4B open model using parameter-efficient QLoRA and compare it against zero-shot, few-shot, and a frontier-model ceiling.
- Build a rigorous, self-written evaluation harness and quantify what each stage of training buys.
- Analyse residual failures to characterise when the model can be trusted.

II. Related Work

Parameter-efficient fine-tuning makes adapting large models tractable on commodity hardware. LoRA [1] freezes the base weights and learns a low-rank update; QLoRA [2] adds 4-bit quantisation of the frozen base, enabling billion-parameter fine-tuning in a few gigabytes of GPU memory. Direct Preference Optimization (DPO) [3] aligns a model to preferred outputs without a separate reward model. At industrial scale, DeepSeek-R1 [4] distills reasoning from a large model into small dense models by supervised fine-tuning on teacher-generated traces—the same teacher-to-student pattern this project applies at a single-GPU scale. We use Qwen3-4B-Instruct [5] as the base model and the Unsloth [6] and TRL [7] libraries for efficient training.

III. Methodology

A. Task Specification

The input is a raw text string; the output is a JSON object with a list of expense objects and an overall confidence label. Each expense has the fields amount, currency (ISO 4217 or UNKNOWN), category (a closed enum), merchant, date (resolved against a reference date supplied in the prompt), description, reimbursable, and policy_flags (a closed set such as over_limit, missing_receipt). At inference the system prompt carries the schema, a reference

date, and a compact policy block; this same prompt is used for every system so that measured differences are attributable to the training method, not prompt wording.

B. Dataset Construction

Because manually labelling thousands of examples is infeasible, we generate data procedurally: each example is composed from orthogonal axes (source type $\times$ language $\times$ ambiguity $\times$ number of expenses $\times$ policy situation), and the gold label is computed from the policy rules in code. Consequently, input and label can never disagree—the label is a computation, not an opinion. A programmatic perturbation pass then injects realism (typos, OCR glyph confusions such as $0 \leftrightarrow O$, and currency-symbol variants) to the input only, preserving the label.

A contamination check using token-set fuzzy matching removes any training example that is a near-duplicate of a test example; on the first pass it culled 1,134 template twins, precisely the leakage it exists to prevent. The final splits are 1,600 train / 100 dev / 200 test, all schema-valid, with zero train–test contamination. During validation, a disagreement with a frontier model surfaced 349 labels naming a merchant absent from the input; these were repaired deterministically. The test set was frozen and used exactly once per system.

C. Model and Fine-Tuning

We fine-tune with QLoRA. The frozen base weights W are stored in 4-bit NF4; a low-rank adapter is learned such that the effective weight is

$$W' = W + \frac{\alpha}{r}BA, \quad A \in \mathbb{R}^{r \times d}, B \in \mathbb{R}^{d \times r}, \quad (1)$$

with rank $r = 16$, scaling $\alpha = 16$, applied to all seven linear projections of the transformer. Only A and B are trained (under 1% of parameters). Training used learning rate 2×10^{-4} , a cosine schedule, two epochs, effective batch size 16 via gradient accumulation, 8-bit AdamW, and gradient checkpointing, on a single free-tier T4 GPU. Validation loss fell from 0.112 to 0.063 with no over-fitting.

D. Evaluation Harness

Evaluation is deliberately hand-written (~ 300 lines) rather than delegated to a framework, and is built before any training so that metric definitions cannot be tuned to flatter the model. Five metric layers each isolate a distinct question:

- Schema validity: does the output parse and satisfy the strict pydantic contract?
- Field accuracy: per-field correctness (amounts within ± 0.01).
- Expense-level F_1 : for multi-expense inputs, are all expenses found with no hallucinated extras?
- Policy accuracy: are reimbursable and the exact set of policy_flags both correct—the reasoning metric.
- Cost/latency: local throughput versus API price.

TABLE I

Results on the 200-example test set. E2[†] is depressed by a decoding artifact (Sec. V-A); the true capability is in Table II.

System	Schema	Exp. F_1	Date	Policy
E0 zero-shot	99.5%	0.980	42.9%	27.1%
E1 5-shot	98.0%	0.988	79.5%	34.3%
E2 SFT [†]	81.0%	0.898	77.9%	45.2%
E4 frontier	100%	1.000	100%	90.4%

TABLE II

Apples-to-apples comparison on the same 162 examples.

System	Exp. F_1	Policy	Date	Category
E0 zero-shot	0.980	24.7%	43%	—
E1 5-shot	0.986	32.4%	77%	—
E2 SFT	1.000	55.5%	96%	99.6%
E4 frontier	1.000	89.9%	100%	100%

A key sub-problem is expense alignment: predicted expenses may be reordered, so before grading fields we match predictions to gold greedily on (amount within tolerance, then category). The harness carries 29 unit tests and a gold-echo integration test (a system returning gold scores exactly 1.0), validating the grader itself.

IV. Experimental Setup

All systems are evaluated on the identical 200-example test set:

- E0: Qwen3-4B-Instruct, zero-shot.
- E1: Qwen3-4B-Instruct, 5-shot.
- E2: E0 + SFT (QLoRA).
- E4: a frontier model, zero-shot—the accuracy ceiling.

Preference tuning (E3, DPO) and quantised GGUF export (E5) are implemented but were not executed to completion due to exhaustion of free-tier GPU quotas; they are discussed as future scope.

V. Results and Discussion

A. A Decoding Artifact, Isolated

On evaluation, E2 produced valid JSON on 162 of 200 examples; the other 38 are an identical failure—the model emits a `<tool_call>` token repeatedly and never answers. The cause is that the base model’s chat template exposes a tool-calling token that greedy decoding occasionally collapses into. This is a decoding artifact, not a learning failure, and is removed by suppressing that token at generation. Because every failed example also scores zero on every field, this single artifact depresses all of E2’s aggregate numbers in Table I.

B. The True Effect of Fine-Tuning

Restricting all systems to the 162 examples E2 answered isolates learned capability from the decoding bug (Table II).

Four observations follow. (1) Extraction is easy; reasoning is hard. Even zero-shot, the base model emits valid JSON 99.5% of the time and finds the right expenses ($F_1 = 0.98$); what it cannot do is apply the policy (27%) or resolve relative dates (43%). (2) Few-shot teaches format, not reasoning. Five in-context examples lifted date resolution (43% $\rightarrow$ 79%) but policy only marginally (27% $\rightarrow$ 34%). (3) SFT teaches the reasoning. On the shared set, fine-tuning more than doubled policy accuracy (24.7% $\rightarrow$ 55.5%), nearly solved date resolution (43% $\rightarrow$ 96%), and made expense-finding perfect—the central result of this work. (4) The model sits between few-shot and the frontier, closing roughly half the policy gap to a frontier model using a 4B model, 1,600 synthetic examples, and no paid training compute. Once deployed on local hardware the marginal inference cost is effectively zero, versus a per-message API charge for the frontier system.

C. Failure Analysis

Aside from the decoding artifact, residual errors among answered examples concentrate in policy edge cases—exactly the 55.5%-versus-90% gap that preference tuning (DPO) is designed to close by training on the model’s own harvested mistakes. This tells a practitioner when to trust the model (clean single-expense inputs) and when to route to a human (ambiguous reimbursability), which is the practical value of rigorous evaluation.

VI. Conclusion and Future Scope

We showed, end to end, that supervised fine-tuning of a 4B open model teaches the policy reasoning and date arithmetic that prompting alone cannot, more than doubling policy accuracy over the base and closing about half the distance to a frontier ceiling—at near-zero inference cost and full data privacy. Equally important, a purpose-built evaluation harness made the result legible and exposed a concrete, fixable decoding failure that aggregate accuracy alone would have hidden.

Future scope. (i) Preference tuning (DPO): harvest the SFT model’s real mistakes into (chosen, rejected) pairs and train with DPO to close the remaining policy gap; the pipeline is implemented. (ii) Quantised deployment: export a 4-bit GGUF and measure the accuracy cost of quantisation and the local-versus-API economics. (iii) Data diversity: blend frontier-generated examples into the procedural corpus to reduce template bias. (iv) Reinforcement learning with a verifiable reward: because our grader is a programmatic reward, GRPO-style training against it is a natural extension.

References

- [1] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” *Int. Conf. Learning Representations (ICLR)*, 2022.
- [2] T. Detrmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [3] R. Rafailov et al., “Direct Preference Optimization: Your Language Model is Secretly a Reward Model,” *NeurIPS*, 2023.
- [4] DeepSeek-AI, “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning,” *arXiv:2501.12948*, 2025.
- [5] Qwen Team, “Qwen3 Technical Report,” *arXiv*, 2025.
- [6] Unsloth AI, “Unsloth: Fast and memory-efficient LLM fine-tuning,” <https://github.com/unslothai/unsloth>, 2024.
- [7] L. von Werra et al., “TRL: Transformer Reinforcement Learning,” <https://github.com/huggingface/trl>, 2020.